FACULTY
OF MATHEMATICS
AND PHYSICS
Charles University

MASTER THESIS

Bc. Ondřej Krsička

Collaborative Editing of Structured Documents in Denicek

Department of Software Engineering

Supervisor of the master thesis:	Mgr. Tomáš Petříček, Ph.D.
Study programme:	Computer Science
Study branch:	Software and Data Engineering

Prague 2026

I declare that I carried out this master thesis on my own, and only with the cited sources, literature and other professional sources. I understand that my work relates to the rights and obligations under the Act No. 121/2000 Sb., the Copyright Act, as amended, in particular the fact that the Charles University has the right to conclude a license agreement on the use of this work as a school work pursuant to Section 60 subsection 1 of the Copyright Act.

In ..... date ....

Author's signature

I would like to thank my supervisor Mgr. Tomáš Petříček, Ph.D. for his guidance and patience throughout this project.

Title: Collaborative Editing of Structured Documents in Denicek

Author: Bc. Ondřej Krsička

Department: Department of Software Engineering

Supervisor: Mgr. Tomáš Petříček, Ph.D., Department of Software Engineering

Abstract: Denicek is a computational substrate for document-oriented end-user programming where users program by recording and replaying edit sequences over tree-structured documents. This thesis evaluates approaches to collaborative editing in Denicek, comparing Conflict-free Replicated Data Types (CRDTs) with the original Denicek's Operational Transformation (OT). We evaluate three approaches — Automerge, Loro, and a custom pure operation-based CRDT — and describe the trade-offs of each with respect to Denicek's requirements: path-based addressing, wildcards, structural edits, and programming by demonstration. The resulting implementation, called mydenicek, stores edits in a grow-only event graph and obtains documents via a deterministic *eval* function that replays events in topological order. Convergence follows from the pure operation-based CRDT framework (G-Set state + deterministic eval); the main engineering contribution is intention preservation — ensuring that references survive structural edits, wildcards expand over concurrent inserts, and recorded edit sequences replay correctly after schema evolution. The implementation is validated on formative examples, 358 tests (including a concurrent pair matrix and property-based convergence testing).

Keywords: CRDT collaborative editing operational transformation local-first software end-user programming Denicek

Název práce: Kolaborativní editace strukturovaných dokumentů v systému Denicek

Autor: Bc. Ondřej Krsička

Katedra: Katedra softwarového inženýrství

Vedoucí diplomové práce: Mgr. Tomáš Petříček, Ph.D., Katedra softwarového inženýrství

Abstrakt: Denicek je výpočetní substrát pro *document-oriented end-user programming*, ve kterém uživatelé programují nahráváním a replayováním editačních sekvencí nad stromově strukturovanými dokumenty. Tato práce evaluuje přístupy ke kolaborativní editaci v systému Denicek, porovnávající *Conflict-free Replicated Data Types* (CRDT) s původní *Operational Transformation* (OT) systému Denicek. Hodnotíme tři přístupy — Automerge, Loro a vlastní *pure operation-based* CRDT — a popisujeme kompromisy každého z nich s ohledem na požadavky systému Denicek: adresování cestou, wildcards, strukturální editace a programování demonstrací. Výsledná implementace mydenicek ukládá editace v grow-only grafu událostí a dokument získává deterministickou funkcí *eval*, která přehraje události v topologickém pořadí. Konvergence plyne z použití *pure operation-based* CRDT (G-Set stav + deterministická eval); hlavním inženýrským přínosem je zachování záměru (*intention preservation*) — zajištění, že reference přežijí strukturální editace, wildcards se rozšíří na konkurentní vložení a nahrané editační sekvence fungují po změně schématu. Implementace je validována na formativních příkladech, 358 testech (včetně matice konkurentních párů a property-based testování konvergence).

Klíčová slova: CRDT, kolaborativní editace, operační transformace, local-first software, programování koncovými uživateli, Denicek

Contents

Introduction **3**
Thesis structure 4

1 Background **5**
1.1 Document-oriented programming 5
1.2 Denicek 5
1.2.1 Edit operations 7
1.2.2 Denicek's collaboration model 8
1.3 Operational Transformation 8
1.4 Causality 9
1.4.1 Event DAG 9
1.5 Reliable causal delivery 9
1.6 Conflict-free Replicated Data Types 10
1.6.1 Pure operation-based CRDTs 11
1.7 Related systems and why we built our own 12

2 Design and implementation **15**
2.1 Document model 15
2.2 Event DAG 17
2.3 Materialization 18
2.3.1 Caching 19
2.3.2 Complexity 19
2.4 Convergence 20
2.5 Edit types 20
2.5.1 Selector rewriting 21
2.5.2 Edit transformation dispatch 21
2.5.3 CopyEdit and mirroring 23
2.5.4 Wildcard edits and concurrent insertions 23
2.6 Implementation details 24
2.6.1 Extensibility 25
2.6.2 Serialization 25

2.6.3 Formula engine 25
2.6.4 Undo and redo 26
2.6.5 Recording and replay 26
2.6.6 Sync and server 26
2.6.7 Web application 28
2.6.8 CI/CD 29

3 Evaluation **31**
3.1 Approach comparison 31
3.2 Formative examples 32
3.2.1 Hello World: custom primitive edits and replay 32
3.2.2 Counter: formulas and programming by demonstration 32
3.2.3 Conference List: adding items with recorded edits . . . 33
3.2.4 Conference Table: structural transformation 34
3.2.5 Conference Table: concurrent editing 35
3.2.6 Button replay after schema evolution 35
3.3 Testing strategy 37
3.4 Property-based tests 38
3.5 Performance 39
3.6 Limitations 40

4 Conclusion **43**
4.1 Discussion 45
4.2 Future work 45

Bibliography **47**

A Implementation repository **51**

Introduction

Document-oriented programming systems [1] are programming environments built around the metaphor of a document editor. The user interacts with a structured document that contains both code and data; they modify the document, trigger computations, and view results — all from a unified interface. Spreadsheets are the most familiar example, but the paradigm also includes data-science notebooks and recent research systems. What distinguishes document-oriented programming from conventional programming is that the program *is* the document: there is no separate source file, build step, or hidden execution state.

Denicek [2] is one such system — a computational substrate for document-oriented end-user programming. Its documents are tagged trees (records, lists, primitives, and references) addressed by path-based selectors such as `speakers/0/name` or `items/*`. Users program by recording edit sequences and replaying them (programming by demonstration). The original Denicek uses Operational Transformation (OT) for collaboration, but OT is notoriously fragile: several published algorithms were later proven incorrect [3], and the number of pairwise rules grows with each new edit type.

When a document-oriented programming system supports collaborative editing, it faces a distributed systems challenge: multiple peers modify the same document concurrently, and the system must merge their edits so that all peers converge to the same state. *Local-first software* [4] makes this explicit — each device holds a full copy, edits apply immediately, and synchronization happens in the background. Two main families of algorithms address convergence: Operational Transformation (OT), which transforms operations pairwise, and *Conflict-free Replicated Data Types* (CRDTs) [5], which structure the data so that concurrent modifications can be merged without conflicts. CRDTs guarantee *strong eventual consistency*: any two replicas that have received the same set of updates will be in the same state.

This thesis investigates more robust collaborative editing for Denicek. We evaluate two CRDT libraries — Automerge and Loro — and build a custom design, describing the trade-offs of each. The result, *mydenicek*, is a **pure operation-based CRDT** [6]: edits are stored in a grow-only event graph, and the document

is computed by a deterministic *eval* function that transforms concurrent edits during topological replay.

Convergence — all peers reaching the same state — is the baseline guarantee. The harder problem is **intention preservation** [7]: the effect of an operation in the merged document should match the effect that the operation had on the document state from which it was generated. Convergence requires that replicas eventually see the same event set and evaluate it deterministically; intention preservation is what makes the merged result *meaningful*. For tree-structured documents with path-based selectors, this means that when the document structure changes concurrently — fields are renamed, nodes are wrapped, items are inserted — each operation must still reach its intended target and produce its intended effect. Defining a clear model of user intent for each class of concurrent interaction, and designing merge rules that preserve it, is the central challenge of this thesis.

The main contributions are:

- Evaluation of document-based representations in two CRDT libraries (Automerge, Loro) for tree-structured collaborative editing, identifying concrete limitations: the concurrent wrap problem and the retargeting problem (Chapter 1).
- A novel pure operation-based CRDT for tagged-tree documents with path-based selectors, wildcards, relative references, and strict indices, together with a proof of strong eventual consistency (Chapter 2).
- Semantics that preserves user intent across concurrent edits, including explicit invariants: wildcard edits affect concurrent insertions, references survive structural changes, and indices shift correctly (Chapter 3).
- Programming-by-demonstration support built on the same underlying event graph: recorded edits are retargeted through later structural changes, enabling replay after schema evolution (Chapter 3).

Thesis structure

Chapter 1 covers CRDTs, OT, causality, and related work. Chapter 2 presents the mydenicek CRDT design and implementation. Chapter 3 evaluates correctness and performance on formative examples. Chapter 4 concludes with future work.

Chapter 1

Background

1.1 Document-oriented programming

Document-oriented programming systems [1] are programming environments built around a document editor. The user interacts with a structured document that contains both code and data; they modify the document, trigger computations, and view results from a unified interface. Spreadsheets are the most widely used example: a spreadsheet is simultaneously a program (formulas), data (cell values), and a user interface (the grid). Other instances include data-science notebooks (Jupyter) and research systems such as Webstrates [8].

Petříček et al. [1] identify twelve design choices that characterize such systems, organized along four axes: document structure and representation, how programming is embedded, how the user interface displays and edits documents, and how computations are evaluated. What distinguishes document-oriented programming from conventional programming is that the program *is* the document: there is no separate source file, build step, or hidden execution state. This thesis adds a fifth concern — *collaborative editing* — and investigates the design choices it introduces for a specific document-oriented system.

1.2 Denicek

Denicek [2] is a document-oriented programming system — a computational substrate for end-user programming. Its documents are *tagged trees* — hierarchical structures where each node carries a structural tag (such as `h1`, `ul`, `tr`) and contains either named fields (records), ordered children (lists), scalar values (primitives), or pointers to other nodes (references). Figure 1 shows an example document tree; each node type is described in more detail in Section 2.1.

The Denicek paper demonstrates the substrate through two systems built

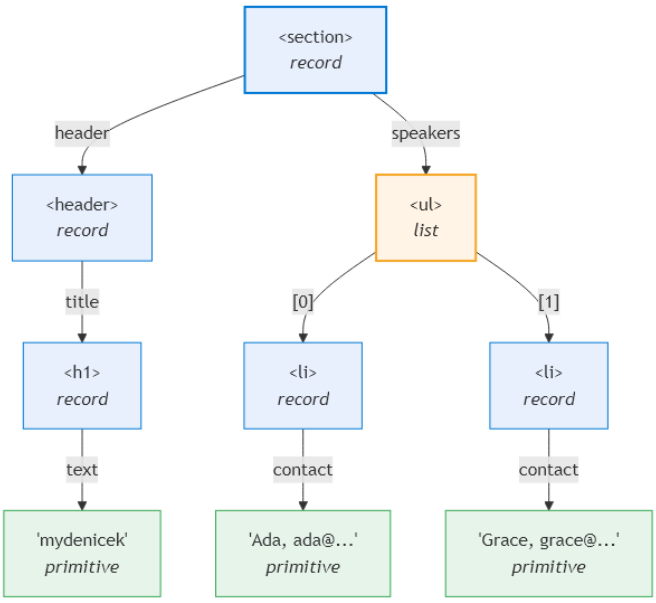

Figure 1 Example Denicek document tree. Blue nodes are records (named fields), orange is a list (ordered children), green nodes are primitives (scalar values). Edge labels show field names and list indices.

on top of it. *Webnicek* is a web-based programming system where documents are rendered as interactive web pages and users program by manipulating the document structure. *Datnicek* is a data science notebook that uses the same substrate for tabular data with formulas, supporting datasets of up to thousands of rows. Both systems share the same edit operations, recording/replay mechanism, and collaboration model — they differ only in how the document tree is rendered and interpreted.

Nodes are addressed by *selector paths* — slash-separated strings that describe the location of a node in the tree (see Figure 1). For example, `/speakers/0/name` refers to the `name` field of the first item in the `speakers` list. Selectors support wildcards: `/speakers/*` addresses all children of the `speakers` list, enabling bulk operations such as "update the tag of every list item."

Denicek provides four key end-user programming experiences:

- **Programming by demonstration.** Users perform edits interactively — such as adding a list item and copying a value from an input field — and the system records these edits as a replayable script. When the user clicks a button, the recorded edits are replayed, potentially on different targets.
- **Schema evolution.** Structural edits allow users to refactor the document's structure without losing data or breaking references.

- **Collaborative editing.** Multiple peers can edit the same document concurrently, and the system merges their edits deterministically. Notably, Denicek supports wildcard selectors (`speakers/*`) that target all children of a node. When combined with concurrent insertions, this produces an unusual semantics: a wildcard edit affects not only items that existed when the edit was made, but also items inserted concurrently by other peers (Section 2.5.4).
- **Formula recomputation.** Nodes can contain formulas that reference other nodes via relative paths. When the referenced data changes, the formula result is recomputed.

1.2.1 Edit operations

Denicek provides two categories of edit operations. *Data edits* modify the content of the document:

- `add(target, field, value)` — add a named field to a record
- `delete(target, field)` — remove a field from a record
- `set(target, value)` — replace a primitive value
- `insert(target, index, item)` — insert an item into a list at a given index
- `remove(target, index)` — remove the item at a given index from a list
- `copy(target, source)` — copy a subtree from one location to another

Structural edits change the *paths* by which nodes are addressed, not just the values they contain:

- `rename(target, oldField, newField)` — rename a record field
- `updateTag(target, newTag)` — change a node's structural tag (e.g., `ul` to `table`)
- `wrapRecord(target, field, tag)` — wrap a node in a new parent record, moving the original value into a named field
- `wrapList(target, tag)` — wrap a node in a new parent list
- `reorder(target, permutation)` — reorder list items by applying a full permutation

The distinction matters for collaborative editing: while data edits modify a node's content at a fixed path, structural edits change the *paths* by which other edits address nodes. When a peer renames `speakers` to `talks`, all concurrent edits targeting `/speakers/...` must be retargeted to `/talks/...`. When a peer wraps a node, concurrent edits must gain an additional path segment. This is the core challenge that any collaborative editing approach for Denicek must solve.

1.2.2 Denicek's collaboration model

The original Denicek defines three core operations on edit histories:

1. **Apply** — apply an edit to a document, producing a new document state and extending the history.
2. **Merge** — merge two edit histories that diverged from a common ancestor, using OT to transform one history's edits against the other's.
3. **Check for conflicts** — after merging, identify edits that could not be reconciled (e.g., concurrent deletion and modification of the same node) and report them to the user.

Denicek's histories are *linear sequences* of edits. Merging works similarly to `git rebase`: given two branches that diverged from a common ancestor, merge transforms one branch's edits through the other's, appending the transformed edits to produce a single linear history. Because the transformation is applied in one direction, merge is *not commutative* — merging history A into B may produce a different result than merging B into A. The paper mentions that histories could form a graph rather than a linear sequence, but does not elaborate on this direction.

This thesis takes exactly that step: replacing linear histories with a *causal event graph* (DAG), where merge order does not matter because all peers replay the same deterministic topological order. The formal framing of this approach as a pure operation-based CRDT is presented in Section 2.4, after the necessary background on CRDTs is introduced in Section 1.6.

1.3 Operational Transformation

Operational Transformation (OT), introduced by Ellis and Gibbs [9], transforms concurrent operations so that both peers converge to the same state (e.g., two concurrent text insertions at the same position are shifted so both apply correctly).

Correctness in the decentralized setting requires two *transformation properties* [10]: **TP1** (applying a then $T(b, a)$ reaches the same state as b then $T(a, b)$) and **TP2** (transforming a third operation through two concurrent ones is order-independent). Several published algorithms were later proven to violate TP2 [3], and the number of pairwise rules grows with each new edit type. The original Denicek's OT probably does not satisfy TP1/TP2 either — the intention was that they might hold for edit sequences without conflicts, but the conflict mechanism was never reliable enough to guarantee this. mydenicek sidesteps TP1/TP2 entirely by using a single deterministic replay order (Section 2.2): since all peers apply edits in the same topological sequence, the symmetric transformation properties are not required.

Sun et al. [7] decompose OT correctness into three properties: **convergence**, **causality preservation**, and **intention preservation** (the effect matches the user's intent). TP1/TP2 address convergence; intention preservation depends on the transformation rules and cannot be derived from them. This thesis proves convergence in Section 2.4 and validates intention preservation empirically through the formative examples of Section 3.2.

1.4 Causality

Collaborative editing requires tracking which edits are aware of which others — the *causal* ordering. Without this, a system cannot distinguish concurrent edits (which need conflict resolution) from sequential ones (which do not). Lamport's *happens-before* relation [11] defines a partial order over events in a distributed system:

> **Definition (happens-before).** For events a and b, $a \to b$ if (1) a and b are events on the same peer and a preceded b, (2) a is the sending event of some message m and b is the receiving event of m, or (3) there exists an event c such that $a \to c$ and $c \to b$ (transitivity). Two events are *concurrent* ($a \parallel b$) if neither $a \to b$ nor $b \to a$.

Vector clocks [12, 13] implement happens-before detection. A vector clock V maps each peer ID to its highest known sequence number. For a set of peers P, $a \to b$ iff $V_a[p] \leq V_b[p]$ for all $p \in P$ with strict inequality for at least one. This allows concurrency detection in $O(|P|)$ time.

1.4.1 Event DAG

Causal relationships form a directed acyclic graph (DAG): each event lists its parents (direct causes). The DAG encodes the full causal structure and enables deterministic replay. The *frontier* — events with no descendants — compactly represents the current state. When a peer creates a new event, the current frontier becomes its parents; after sync, a new event merges multiple parents back to a single point, analogous to a merge commit.

1.5 Reliable causal delivery

The pure op-based CRDT framework [6] requires two delivery guarantees:

Causal consistency. If operation a is delivered, all operations causally before a have already been delivered.

Eventual delivery. Every operation generated by a correct process is eventually delivered to all correct processes.

In mydenicek, causal delivery is implemented by buffering: when a message arrives whose parents have not yet been delivered, it is held until they arrive. Eventual delivery is ensured by frontier-based catch-up: on each sync round, a peer sends its frontier, and the server responds with all missing events. If a message is lost, the frontier does not advance and missing events are resent on the next round.

1.6 Conflict-free Replicated Data Types

CRDTs [5] are data structures designed for distributed systems where multiple replicas can be modified independently and merged without conflicts. The key guarantee is *strong eventual consistency*: any two replicas that have received the same set of updates will be in the same state, regardless of the order in which updates were delivered. Preguiça [14] provides a comprehensive overview of CRDTs and their variants.

CRDTs come in two main flavors:

- **State-based CRDTs** (CvRDTs) require the set of possible states to form a *join semilattice* — a partially ordered set where any two states have a least upper bound (join). Replicas periodically send their full state to each other, and the merge operation computes the join. This works over unreliable channels since states can always be re-merged, but sending the full state can be expensive for large data structures.
- **Operation-based CRDTs** (CmRDTs) propagate individual update operations rather than full states. Concurrent operations must be commutative so that applying them in either order produces the same result. This is more bandwidth-efficient, but requires a reliable causal delivery layer as described in Section 1.5.

A hybrid approach, *delta-state CRDTs*, sends only the part of the state that changed (the "delta") rather than the full state. Deltas are joinable like full states (so they tolerate message loss and reordering) but are small like operations (so they are bandwidth-efficient).

1.6.1 Pure operation-based CRDTs

Pure operation-based CRDTs [6] are the theoretical foundation of mydenicek. Traditional operation-based CRDTs require concurrent operations to commute pairwise, which is difficult to design and prove for complex data types. Baquero et al. sidestep this by making the **replica state the set of all delivered operations** (a *PO-Log* — partially ordered log). The observable value is computed on demand by a deterministic function *eval* over that set.

The framework organizes the lifecycle of an operation into three phases: **prepare** takes a user edit and the current document state and creates an operation annotated with causal metadata — a vector clock that records which operations the creator had already seen (e.g., "insert item X at position 3, with clock {alice: 5, bob: 3}"); **effect** adds the operation to the PO-Log on the local replica and broadcasts it to remote replicas; **eval** computes the observable document by applying all operations in the PO-Log. The PO-Log is a G-Set (grow-only set, see below) whose merge is set union — associative, commutative, and idempotent. Shapiro et al. [5] proved that state-based CRDTs with a monotonic join-semilattice merge converge; the G-Set satisfies this trivially. Strong eventual consistency follows from two conditions: (1) every operation is eventually delivered to every replica (*eventual delivery*), and (2) *eval* is deterministic. Condition (1) is a transport-layer concern; condition (2) is the only property the data type designer must prove.

The framework assumes *reliable causal broadcast* [6, §3]: delivery is causal (if $a \to b$, then a is delivered before b), and every operation is eventually delivered. It also defines a mechanism for pruning *causally stable* operations (delivered to all replicas) from the PO-Log.

mydenicek implements this framework directly. The mapping is:

Baquero	mydenicek
PO-Log	Event DAG stored as per-peer arrays of events, each event having a list of parent references
prepare	`Denicek.add()`, `.insert()`, etc.
effect	`EventGraph.insertEvent()`
eval	`EventGraph.materialize()`
Causal broadcast	WebSocket relay + causal delivery buffer
PO-Log pruning	Not feasible on relay server (see below)

The event DAG is strictly richer than a PO-Log: it stores explicit parent pointers, enabling incremental materialization. PO-Log pruning (compaction) is not implemented — the relay server operates in append-only mode, and history compaction is left for future work (Section 4.2).

Common CRDT building blocks relevant to this thesis include:

- **G-Set** (grow-only set): elements can be added but never removed. Merging two G-Sets is simply their union.
- **LWW-Register** (last-writer-wins register): a single value where concurrent writes are resolved by a deterministic ordering (typically by timestamp or logical clock).
- **OR-Set** (observed-remove set): elements can be added and removed. Concurrent add and remove of the same element are resolved in favor of the add.

For collaborative editing of tree-structured documents, Kleppmann and Beresford [15] proposed a JSON CRDT that uses unique identifiers for each node and supports insert, delete, and move operations. This work identified the *move operation problem*: in a flat JSON structure without native move support, moving a node requires deleting it from one location and inserting it at another — two separate operations that can interleave with concurrent edits, potentially losing data. For collaborative *rich text* (text with formatting spans and block structure), Peritext [16] extends character-level CRDTs with mark operations whose semantics tolerate concurrent insertions and deletions within the affected range — a problem closely related to the "wildcard affects concurrent inserts" behavior we describe in Section 2.5.4.

1.7 Related systems and why we built our own

Several existing CRDT libraries were evaluated as potential backends for Denicek.

Automerge [17] provides JSON-like data structures (maps, lists, text, counters) with automatic conflict resolution. We represented Denicek's tagged trees as a flat map of nodes with parent-child references. This worked until we encountered the **concurrent wrap problem**: wrapping a node in a new parent requires removing it from the old parent and inserting it into a new one — two non-atomic Automerge operations. When two peers concurrently wrap the same node, the node ends up referenced from two parents, breaking the tree invariant. The deterministic-ID workaround was fragile and did not scale to nested wraps.

Loro [18] solved the wrap problem with a native movable-tree CRDT [19] where moves are atomic. However, Loro uses opaque node IDs rather than path-based addressing. This caused the **retargeting problem**: Denicek's programming by demonstration relies on replaying recorded edits that create nodes with relative references to their neighbors (e.g., a formula cell referencing `../../0/contact/source` — the sibling cell in the same row). With Loro's opaque IDs, such relative references cannot be expressed — the recorded edit can only

reference the specific node ID from the original recording, not the structurally equivalent neighbor in a different row. When the same edit sequence is replayed on a different subtree, the reference points to the wrong node.

Why not layer path OT on top of Loro? Layering creates two independent conflict-resolution regimes. Loro resolves concurrent moves by its internal CRDT semantics; a path-OT layer resolves conflicts by rewriting selectors. When they disagree on the same concurrent edit — which happens precisely in Denicek's core use case of wildcard structural transformations with concurrent inserts — the path layer retargets selectors to paths that Loro's resolution left empty. This mismatch is systematic, not a corner case. We chose to build a unified system where path-based selectors are the native addressing mode.

Other systems. Yjs [20] shares Automerge's limitations (no atomic move, no path-based selectors). Eg-walker [21] stores operations in a causal event graph with topological replay — mydenicek borrows this architectural idea but applies it to trees with selector rewriting. Diamond Types [22] and json-joy [23] target text and JSON respectively but lack wildcard selectors. Weidner [24] identifies the *for-each problem*: operations applied to every element in a range miss concurrently inserted elements. mydenicek's wildcard selectors solve this without a dedicated for-each operation (Section 2.5.4).

Pure op-based CRDT frameworks. Bauwens and Gonzalez Boix have developed an ecosystem for pure operation-based CRDTs: Flec [25] implements PO-Log-based CRDTs with nesting and composition; related work addresses metadata reduction via causal stability [26] and automated verification via VeriFx [27]. Weidner et al. [28] provide Collabs, a CRDT framework with a `TestingRuntimes` harness for deterministic network interleaving. mydenicek does not currently build on these frameworks. Integrating with them — reusing their PO-Log management and metadata reduction while retaining mydenicek's domain-specific selector rewriting rules — is left for future work (Section 4.2).

Chapter 2

Design and implementation

mydenicek is a pure operation-based CRDT for collaborative editing of tagged document trees. The replica state is a grow-only set of edit events; the document is computed by a deterministic *eval* function (`materialize`) that replays events in topological order, transforming edits through concurrent structural changes. Figure 2 shows the data flow. The following sections describe the core CRDT design — the document model, event DAG, materialization algorithm, convergence argument, and edit transformation rules — followed by implementation details of the complete system.

2.1 Document model

Documents are modeled as tagged trees with four node types:

- **Record** — a set of named fields, each containing a child node, plus a structural tag.
- **List** — an ordered sequence of child nodes with a structural tag.
- **Primitive** — a scalar value: string, number, or boolean.
- **Reference** — a pointer to another node via a relative or absolute path.

Reference nodes are unusual in collaborative editing systems. In most CRDT-based systems (Automerge, Loro, json-joy), nodes reference each other via opaque unique IDs — stable across moves and structural changes, but unable to express relative relationships like "my sibling at index 0." In mydenicek, references are *path-based* (`../0/source`), meaning they navigate the tree relative to their position. This enables patterns like formula nodes referencing sibling data, but requires edit transformation to keep references valid when structural edits (rename, wrap) change the paths they traverse.

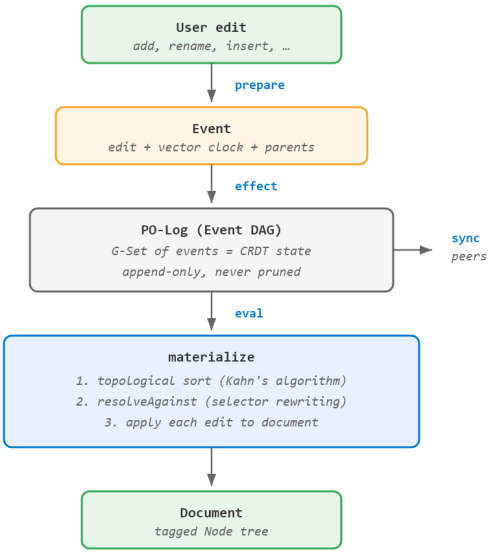

Figure 2 Data flow in mydenicek. A user edit is prepared into a tagged Event, added to the PO-Log (a G-Set of events = the CRDT state), and synced to peers. The *eval* function (`materialize`) sorts events topologically, transforms concurrent edits, and applies each edit to produce the document.

Nodes are addressed by *selectors* — slash-separated paths that describe how to navigate the tree from the root. The selector `speakers/0/name` navigates to the `speakers` field, then to the first list item (index 0), then to the `name` field. Unless stated otherwise, examples in this thesis use selectors *relative to the document root*, without a leading `/`. A leading `/` is only significant in reference nodes (see `Reference` above), where it distinguishes absolute paths from paths relative to the reference's own position. Selectors support three special forms:

- **Wildcards** (`*`): `speakers/*` expands to all children of the `speakers` list. An edit targeting `speakers/*` is applied to every item.
- **Negative indices** (`-1`, `-2`): end-relative list addressing, added in mydenicek. `-1` means the last position (append for insert, last item for remove), `-2` means second-to-last, and so on. Resolved to absolute positions at replay time using a stored `listLength`.
- **Strict indices** (`!0`): `speakers/!0` refers to the item at index 0 *at the time of the edit*, also added in mydenicek. Unlike plain `0`, strict indices are not shifted by concurrent insertions or removals. Their use is described in Section 2.6.5.
- **Parent navigation** (`..`): used in references to navigate up the tree. `../../0/contact` goes up two levels, then navigates to `0/contact`.

Reference nodes (described as the fourth node type above) store their target as a selector path. For example, a reference `"../0/source"` means "navigate up one level from this reference node's position, then down to `0/source`."

Table 1 summarizes the selector notation used throughout this thesis.

Table 1 Selector notation summary.

Form	Example	Meaning
Field name	`speakers/0/name`	Navigate by field name or list index
Wildcard	`speakers/*`	Expand to all children of the target node
Negative index	`insert(items, -1, ...)`	End-relative: `-1` = last position, `-2` = second-to-last
Strict index	`speakers/!0`	Not shifted by concurrent inserts/removes; survives recording artifact removal
Absolute reference	`/speakers/0`	Starts with `/`; resolved from document root
Relative reference	`0/source`	Resolved from the reference node's own position
Relative reference to parent	`../0/source`	Navigate up the tree with `..`

2.2 Event DAG

The event directed acyclic graph (DAG) is the core data structure of mydenicek. It is a grow-only, append-only structure — events are immutable once created, and new events can only be added, never modified or removed. This makes the event set a G-Set (grow-only set), one of the simplest CRDTs: two peers that have received the same set of events will always produce the same document. Each edit creates an *event* containing:

- **EventId** — a unique identifier `peer:seq`, where `peer` is the peer's string identifier and `seq` is a monotonically increasing sequence number. For example, `alice:3` is Alice's third event.
- **Parents** — the set of event IDs that formed the *frontier* of the creating peer's DAG immediately before this event was appended. Equivalently, they are the most recent events the peer had seen. After the new event is inserted, those parents are no longer on the frontier — the frontier collapses to this

new event alone. An event with multiple parents therefore corresponds to the first edit after a sync that brought in another peer's branch, and is the merge point of the two branches.

- **Edit** — the actual edit operation (add, delete, rename, set, insert, wrapRecord, etc.) with its target selector and arguments.
- **Vector clock** — a map from peer ID to the highest sequence number seen from that peer. The vector clock enables causal ordering: event A *happens-before* event B if A's vector clock is dominated by B's. Two events are *concurrent* if neither dominates the other.

Parents and vector clocks serve complementary roles.Parents define the direct edges of the DAG — they are needed for topological sorting and for the sync protocol's `eventsSince(frontiers)` computation. Vector clocks are an optimization: they enable O(P) concurrency detection (where P is the number of peers) during `resolveAgainst`, which must classify every prior event as either a causal ancestor (skip) or concurrent (transform). Without them, the same information could be computed by traversing the DAG to test reachability, but at greater cost. For example, Alice's event with clock `{alice: 5, bob: 3}` and Bob's event with clock `{alice: 2, bob: 4}` are concurrent because neither clock dominates the other (`alice: 5 > 2` but `bob: 3 < 4`).

2.3 Materialization

To reconstruct the document from the event DAG, we perform *deterministic topological replay*:

1. **Order.** Compute a total order consistent with the causal partial order, using `EventId` lexicographic comparison as tie-breaker for concurrent events.
2. **Resolve and apply.** Starting from the initial document, apply each event's edit in order. Before applying, call `resolveAgainst`, which finds all previously applied *concurrent* edits and transforms the current edit's selector through each of them. Events are stored per peer; since sequence numbers are contiguous, the first concurrent event from peer Y is at position $V_E[Y] + 1$ — an $O(1)$ lookup. The concurrent events from all peers are then iterated in topological order, transforming the edit through each one.
3. **Conflicts.** If a transformed edit becomes invalid (e.g., it targets a node deleted by a concurrent edit), it becomes a *no-op conflict* — recorded but not applied. The original event remains in the DAG (events are immutable).

Because the sort order and the selector-rewriting transformations are both deterministic, any two peers that have received the same set of events produce

the same document. This is the strong eventual consistency guarantee, stated precisely in Section 2.4.

2.3.1 Caching

When a new event's parents exactly match the current frontier (the common case during local editing), the event is a *linear extension* of the graph. In that case, `resolveAgainst` is a no-op (every prior is a causal ancestor), so the edit is applied directly to the cached document in $O(S)$ time (where S is the number of nodes matched by the edit's selector) — no topological sort, no concurrency scan. When another peer's incoming event invalidates the linear cache, the materializer rematerializes from scratch.

2.3.2 Complexity

The event DAG under the happens-before relation is a **partially ordered set** (poset). Events from the same peer are totally ordered by sequence number, forming a **chain**. Two events are **comparable** (one is an ancestor of the other) or **incomparable** (concurrent). Let N be the total number of events. **We treat the number of peers P as a constant** — Denicek targets small-group collaboration (typically 2–5 peers), so all $O(P)$ factors reduce to $O(1)$ in the analysis below.

Materialization has two phases:

- **Topological sort.** Kahn's algorithm produces a total order in $O(N + E)$ where E is the number of parent edges. Each event has at most P parents, so $E \leq NP = O(N)$. When multiple events have indegree zero (i.e., they are concurrent), ties are broken by `EventId` comparison using a binary heap. Since each peer's events form a chain, at most P events can have indegree zero at any time, so the heap never exceeds P entries and each operation is $O(1)$. Cost: $O(N)$.

- **Replay.** Each event's edit is applied in order, after transforming it through all incomparable predecessors. Each pairwise transformation is $O(1)$ (fixed dispatch on edit type). Since events are stored per peer in an array indexed by contiguous sequence number, finding the concurrent boundary is $O(1)$ per peer, $O(P) = O(1)$ total. Cost: $O(N + C_{\text{total}})$, where C_{total} is the number of incomparable pairs in the poset — the total number of pairwise transformations actually performed.

The overall cost is $O(N + C_{\text{total}})$.

C_{total} depends on the DAG shape. For a fully sequential chain, $C_{\text{total}} = 0$. For a fork into two branches of lengths a and b, every event in one branch is

incomparable with every event in the other, so $C_{\text{total}} = a{\cdot}b$. For an m-way fork with branches $a_1, \ldots, a_m$: $C_{\text{total}} = \sum_{i<j} a_i \cdot a_j$. The benchmarks in Section 3.5 confirm these predictions: the local-append benchmark (no concurrency) scales linearly in N, while the equal-branch concurrent benchmark scales with $C_{\text{total}} = (N/2)^2$.

2.4 Convergence

mydenicek is a *pure operation-based CRDT* [6] (see Section 1.6.1). The replica state is a grow-only set (G-Set) of events. The document is produced by `materialize` — a deterministic function from the event set to the document tree. Convergence follows from two properties:

1. **Agreement on state.** The G-Set guarantees that any two replicas that have communicated (directly or transitively) hold the same event set (Section 2.6.6).
2. **Deterministic eval.** Given the same event set, `materialize` produces the same document. This holds because each step is deterministic: `topologicalOrder` uses `EventId` comparison (a strict total order), `resolveAgainst` is a sequential walk over the sorted events, and `apply` performs local mutations with no randomness.

Together, these give **strong eventual consistency**: any two replicas that have received the same set of events are in the same state.

Convergence is the easy part. The hard part is **intention preservation** — ensuring that the *result* of merging concurrent edits matches what users intended. References must survive structural edits, wildcards must expand over concurrent inserts, indices must shift through concurrent modifications, and recorded edits must replay after schema evolution. Convergence is guaranteed by the framework; intention preservation is validated empirically through the formative examples (Section 3.2) and property-based tests (Section 3.4).

2.5 Edit types

The system supports 11 edit types: record operations (`RecordAdd`, `RecordDelete`, `RecordRename`), list operations (`ListInsert`, `ListRemove`, `ListReorder`), structural operations (`UpdateTag`, `WrapRecord`, `WrapList`), `CopyEdit` (subtree copy with mirroring), and `ApplyPrimitiveEdit` (extensible custom edits). Three inverse types (`UnwrapRecord`, `UnwrapList`, `RestoreSnapshot`) are produced only by `computeInverse()` for undo.

2.5.1 Selector rewriting

When two edits are concurrent, the later one's selector must be rewritten through the earlier one's structural effect. Table 2 summarizes the rules.

Table 2 Selector rewriting rules.

Edit	Rule	Example
Rename a to b	a/… becomes b/…	speakers/0/name becomes talks/0/name
WrapRecord(f)	a/… becomes a/f/…	x/value becomes x/inner/value
WrapList	a/… becomes a/*/…	x/data becomes x/*/data
Delete	a/… removed	concurrent edit becomes no-op
Insert at i	indices $\geq$ i shift +1	items/3 becomes items/4
Remove at i	index i removed; indices $>$ i shift -1	items/3 becomes items/2
Reorder(f, t)	f becomes t; range shifts	items/1 becomes items/3

Transformations compose sequentially: if Alice renames `speakers -> talks` and Bob wraps each item in a ‹`tr`› record, then Carol's selector `speakers/0/name` becomes `talks/0/name` (through rename), then `talks/0/value/name` (through wrap).

Concurrent conflicts are resolved deterministically: concurrent renames compose (both apply in replay order), concurrent wraps produce double nesting, concurrent list inserts and removes shift through each other, and double removes produce a no-op conflict.

2.5.2 Edit transformation dispatch

A naive implementation requires n^2 pairwise rules for n edit types. mydenicek avoids this with two virtual methods in the `Edit` base class (Figure 3):

- `transformSelector(sel)` rewrites a selector through this edit's structural effect. Non-structural edits return the selector unchanged.
- `transformLaterConcurrentEdit(concurrent)` returns a transformed version of a concurrent edit. The default delegates to `transformSelector`, handling most edit pairs.

This gives n methods instead of n^2: each edit type implements `transformSelector` once, and it works for all concurrent edit types.

Two cases require more than selector rewriting:

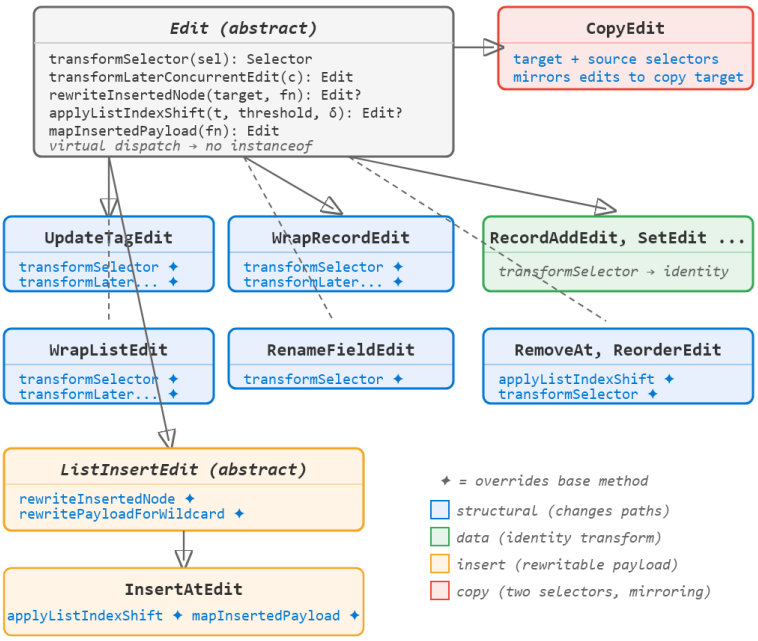

Figure 3 Edit class hierarchy. Structural edits (blue) override `transformSelector`. Data edits (green) return the identity. Insert edits (orange) override payload-rewriting methods. CopyEdit (red) mirrors concurrent edits to both target and source.

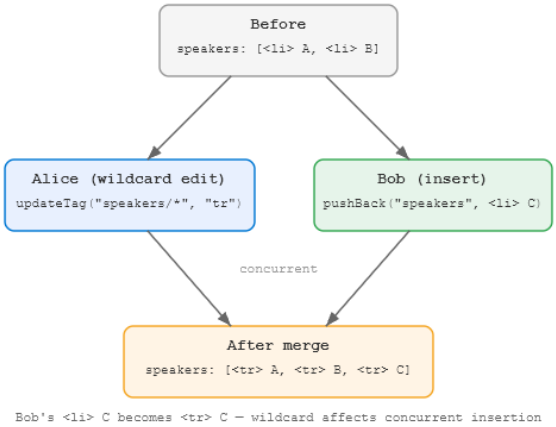

Figure 4 Wildcard edit and concurrent insertion. Alice's wildcard `updateTag` and Bob's `insert` are concurrent. After merge, Bob's inserted `‹li› C` becomes `‹tr› C`.

- **Payload rewriting.** A structural wildcard edit concurrent with a list insert must modify the *inserted node*, not just the insert's selector. Structural edits call `concurrent.rewriteInsertedNode(target, rewriteFn)` — a virtual method overridden by `ListInsertEdit` to apply the rewrite to its payload.
- **Index shifting.** List edits that shift indices call a virtual method on the concurrent edit, passing the target, threshold, and delta. Each list edit type shifts its own indices. This replaces $3 \times 3 = 9$ pairwise rules with three single-method overrides.

2.5.3 CopyEdit and mirroring

`CopyEdit` has two selectors — `target` and `source`. When a concurrent edit modifies the source, the same modification is replicated onto the copy target by wrapping the concurrent edit in a `CompositeEdit` that applies to both locations.

This differs from payload rewriting (Section 2.5.2): payload rewriting changes *what gets inserted*, mirroring duplicates *where an edit applies*.

2.5.4 Wildcard edits and concurrent insertions

Wildcard edits (e.g., `updateTag("speakers/*", "tr")`) affect concurrently inserted items — a deliberate design choice matching the original Denicek semantics, not a consequence of convergence (Figure 4).

The mechanism depends on replay order: if the insert replays first, the wildcard naturally expands to include it; if the wildcard replays first, the insert's payload is rewritten to match. Both orders produce the same result. This is

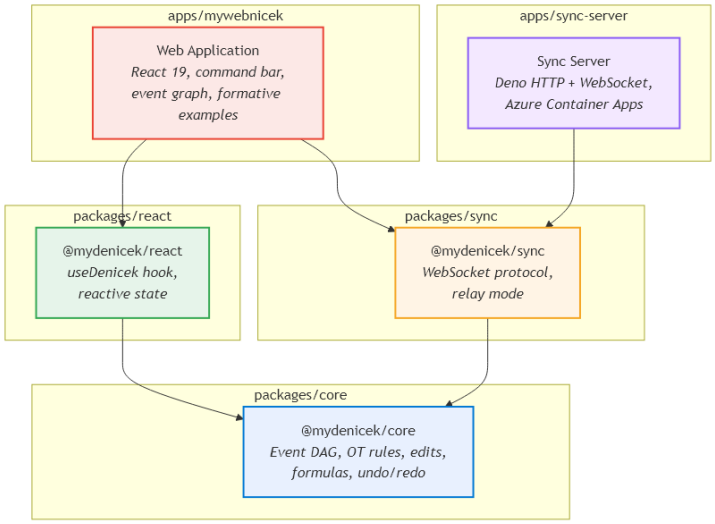

Figure 5 Architecture of the mydenicek monorepo. The core engine is transport-agnostic; the sync server operates in relay mode.

uncommon in CRDTs — most systems only affect items that existed when the operation was created. Weidner [24] calls this the *for-each* problem. In mydenicek, wildcards are expanded at replay time, not creation time, so the for-each semantics follows naturally.

2.6 Implementation details

The implementation is a Deno/TypeScript[1] monorepo published on JSR[2], organized in three packages (Figure 5): **`@mydenicek/core`** (the CRDT engine — pure TypeScript, single runtime dependency), **`@mydenicek/react`** (React bindings), and **`@mydenicek/sync`** (WebSocket relay). Two applications use them: a web frontend (**`apps/mywebnicek`**) and a deployed sync server (**`apps/sync-server`**). The core has no knowledge of the transport layer; the server has no knowledge of edit types.

Table 3 shows the size of each component. The API documentation is published at `https://jsr.io/@mydenicek/core/doc`.

[1]Deno (`https://deno.com`) is a TypeScript/JavaScript runtime with built-in tooling (formatter, linter, test runner, type checker).

[2]JSR (`https://jsr.io`) is a TypeScript-first package registry with Sigstore provenance attestation.

Table 3 Lines of TypeScript (non-empty, non-comment).

Component	Source	Tests
`@mydenicek/core` (CRDT engine)	5,770	6,898
`@mydenicek/sync` (WebSocket relay)	1,050	673
`@mydenicek/react` (React bindings)	364	—
Web application	2,775	—
Sync server	558	—
Total	**10,517**	**7,571**

2.6.1 Extensibility

The core engine is extended via two registries:

- **Primitive edits.** Applications register custom transformations on primitive values via `registerPrimitiveEdit(name, fn)` — for example, `splitFirst` and `splitRest` for the conference table.
- **Formula operations.** Custom formula evaluators are registered via `registerFormulaOperation` and `registerTagEvaluator`.

Both are stored by name in the event DAG and replayed on all peers. The sync server does not need to know about them.

2.6.2 Serialization

For wire transport and persistence, events are serialized as JSON using a codec layer. Each `Edit` subclass implements `encodeRemoteEdit()` to produce its serialized form. Decoding uses a registry: each edit type registers a decoder function keyed by its `kind` string, avoiding a central switch statement.

2.6.3 Formula engine

Denicek's document model includes formula nodes — records whose values are computed from other parts of the document rather than stored directly. mydenicek supports two kinds of formulas: **tag-based** (a `RecordNode` with a specific tag, e.g., `split-first`) and **operation-based** (a `RecordNode` with tag `x-formula` and an `operation` field). References in formula arguments are resolved transparently; circular references are detected by a visited-set check.

2.6.4 Undo and redo

Each `Edit` implements `computeInverse(preDoc)` returning the inverse edit (e.g., `RecordAddEdit -> RecordDeleteEdit`). Undo creates a new event containing the inverse, computed against the document state at the original event's parent frontier. This event syncs to all peers like any other edit.

2.6.5 Recording and replay

Programming by demonstration stores event IDs as replay steps (typically in a button node). On replay, the system captures the source event's edit and transforms its selector through every later edit in the topological order — the same edit transformation pipeline used for concurrent resolution. The replayed edit behaves as if performed concurrently with all events since recording: the transformations retarget it through every structural change. Strict indices (`!0`) pin a selector to a fixed position so it is not shifted by later list insertions or removals.

2.6.6 Sync and server

Convergence requires only that all peers eventually receive the same event set. mydenicek uses a centralized sync server (`apps/sync-server`) operating in **relay mode** (Figure 6) — it stores and forwards events without materializing the event graph into a document. The server organizes collaboration into *rooms* — each room holds the initial document, the event DAG, and the set of connected peers. Both sides exchange **sync** messages containing their current frontiers and any events the other side is missing.

The server operates in relay mode: events are persisted to append-only ND-JSON files. Reliability is achieved through frontier-based catch-up: dropped connections are recovered by resending missing events on reconnection. Remote events are accepted without edit validation — each peer validates locally before sending. The system assumes a trusted peer set and does not defend against Byzantine faults.

The sync server is deployed on Azure Container Apps. We chose Container Apps over both serverless platforms (Azure Functions, AWS Lambda) and traditional PaaS (Azure App Service): serverless platforms impose per-request timeouts that break long-lived WebSocket sessions, while App Service's free F1 tier caps concurrent WebSocket connections at five[3] and paid tiers run the container continuously even when no peers are connected. Container Apps runs long-lived

[3]The free F1 tier of App Service is limited to 5 concurrent WebSocket connections per instance, see `https://learn.microsoft.com/en-us/azure/azure-resource-manager/management/azure-subscription-service-limits#azure-app-service-limits`.

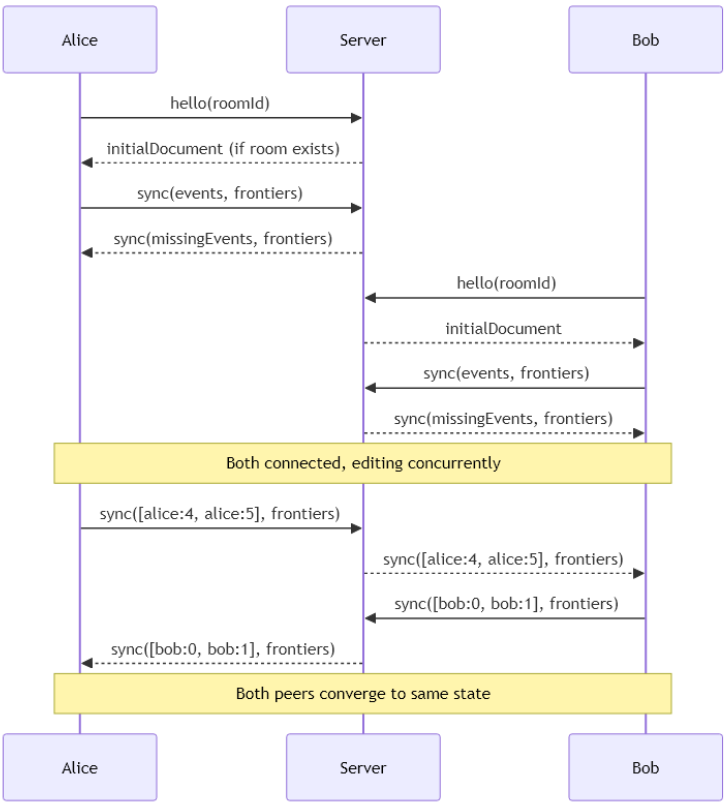

Figure 6 Sync protocol sequence diagram.

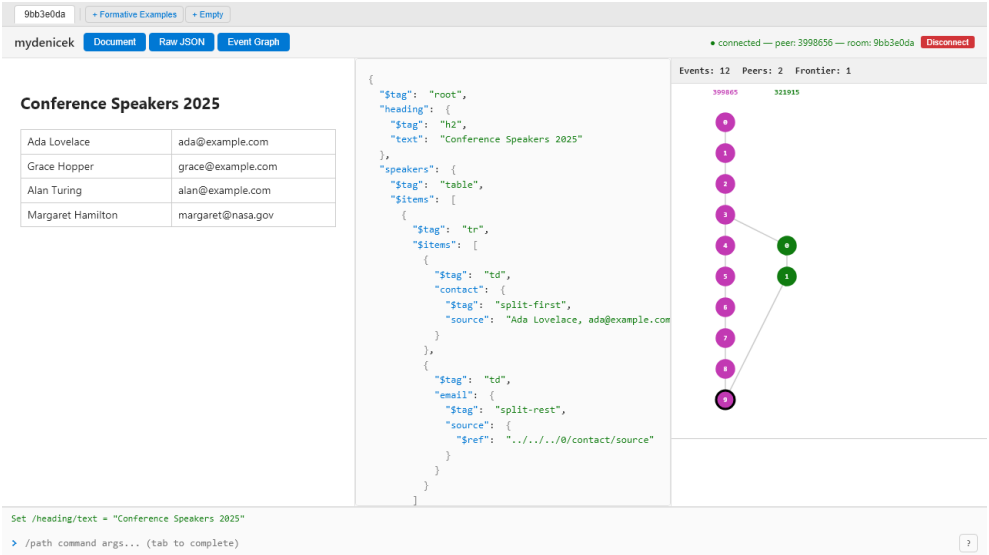

Figure 7 The mydenicek web application after merging concurrent edits. Left: rendered conference table. Center: raw JSON. Right: event DAG showing a fork-and-merge.

processes without request timeouts and scales to zero when idle, so the sync server incurs no compute cost during periods without active peers. The NDJSON event log is persisted to a mounted Azure Files share at `/mnt/sync-data`, since the container's local filesystem is ephemeral and would be lost on each restart or scale-to-zero event. The main drawback is cold-start latency: the first peer to connect after a period of inactivity waits a few seconds while the container is pulled and started.

2.6.7 Web application

The web application (`apps/mywebnicek`) is a single-page React application. The interface provides three panels (Figure 7): rendered HTML, raw JSON, and an event DAG visualization. A command bar executes edits via `/selector command args` syntax.

The web application is built into a static bundle and served from GitHub Pages, which is sufficient because the frontend is fully client-side — it talks directly to the Azure-hosted sync server over WebSocket and needs no application server of its own. The source code is at `https://github.com/krsion/mydenicek` and the live demo at `https://krsion.github.io/mydenicek`.

2.6.8 CI/CD

Two GitHub Actions pipelines automate testing and deployment. The continuous-integration pipeline runs on every push to the main branch: it executes the full test suite (unit, property-based, and concurrent-pair-matrix tests) for `@mydenicek/core` and `@mydenicek/sync`, type-checks all packages, builds the web frontend, and on success deploys the frontend to GitHub Pages and the sync server image to Azure Container Apps via an OIDC-authenticated workflow (no long-lived secrets in the repository). A separate release pipeline, triggered manually, publishes the public packages (`@mydenicek/core`, `@mydenicek/react`, `@mydenicek/sync`) to JSR with Sigstore provenance attestation, linking each published package to the exact source commit and workflow run that produced it.

Chapter 3

Evaluation

This chapter evaluates the mydenicek CRDT along three axes: *does it meet Denicek's requirements* (approach comparison), *does it produce correct results* (formative examples, testing, property-based tests), and *is it fast enough* (performance). We begin with the approach comparison that motivated the custom design, then demonstrate correctness through formative examples that exercise the key features, and conclude with quantitative evaluation.

3.1 Approach comparison

Table 4 summarizes the three approaches evaluated in this thesis against Denicek's key requirements.

Table 4 Comparison of the three approaches against Denicek's requirements.

Requirement	Automerge	Loro	mydenicek (custom)
Atomic move/wrap	No (two-step)	Yes (movable tree)	Yes (selector rewriting)
Path-based addressing	No (opaque IDs)	No (opaque IDs)	Yes (native)
Wildcard selectors	No	No	Yes
Relative references	No	No	Yes (path-based)
Replay retargeting	No	No (ID-based)	Yes (selector rewriting)

Requirement	Automerge	Loro	mydenicek (custom)
For-each semantics	No	No	Yes (wildcard expansion)
Character-level text	Yes	Yes (Fugue)	No (LWW)

Automerge and Loro excel at general-purpose collaborative JSON editing but lack the path-based features Denicek requires. The custom approach sacrifices character-level text editing but gains native support for all of Denicek's programming-by-demonstration features. The original Denicek uses a Git-like model where peers work on local branches and merge manually; mydenicek replaces this with automatic merge via deterministic replay.

3.2 Formative examples

The following examples demonstrate that mydenicek meets each requirement from Table 4. Each example is implemented as a test that constructs documents, applies edits, syncs between peers, and asserts the expected result.

3.2.1 Hello World: custom primitive edits and replay

The first example demonstrates *custom primitive edits* and *wildcard replay*.

```
registerPrimitiveEdit("capitalize", value -> titleCase(value))

e = recordedPeer.applyPrimitiveEdit("messages/0", "capitalize")
sync(recordedPeer, replayPeer)
replayPeer.replay(e, "messages/*")

Before:  messages = ["hello WORLD", "fOO bar"]
After:   messages = ["Hello World", "Foo Bar"]
```

The CRDT is extensible — users register domain-specific transformations that participate in the event DAG and can be replayed with wildcards.

3.2.2 Counter: formulas and programming by demonstration

The Counter example demonstrates the *formula engine* and *recording/replay* (programming by demonstration). Figure 8 shows the counter after one increment.

Figure 8 Counter example: the value 1 is computed by the `x-formula-plus` formula node wrapping the original 0. The "Increment" button replays three recorded edits.

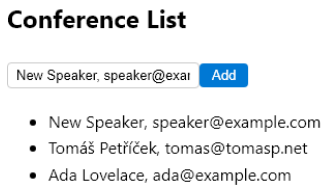

Figure 9 Conference list example: an input field, an "Add" button, and a bullet list of speakers. The button replays two recorded edits (insert + copy from input).

The document starts with `counter/value = 0`. Three recorded edits implement "increment": wrap the value into a `plus(_, 1)` formula node, rename the inner field, and add the right operand:

```
e1 = wrapRecord("counter/value", field="value", tag="plus")
e2 = rename("counter/value", "value" -> "left")
e3 = add("counter/value", "right", 1)
button.steps = [e1, e2, e3]

Before:  counter = 0
After:   counter = plus(0, 1) -> 1
Replay:  counter = plus(plus(0, 1), 1) -> 2
```

Each replay re-executes all three recorded edits (wrap, rename, add). Their selectors are internally transformed through all prior structural edits to match the original value's new location inside the nested wrappers. The formula engine evaluates the nested structure recursively.

3.2.3 Conference List: adding items with recorded edits

The conference list demonstrates recorded edits with an input field and a button. Figure 9 shows the rendered list.

Two edits are recorded: insert a new empty item, then copy the input field value into it. The copy uses a strict index (`!0`) so that during replay, `resolveAgainst` does not shift it to the item that was added by the first recorded insert:

```
e1 = insert("items", index=0, value=<li text="">)
e2 = copy("items/!0/text", from="input/value")
button.steps = [e1, e2]
```

Conferences

Jan Novák, jan@novak.cz | Add Speaker

Jan Novák	jan@novak.cz
Tomáš Petříček	tomas@tomasp.net
Ada Lovelace	ada@example.com

Figure 10 Conference table after structural transformation: names and emails are split into separate columns using `split-first` and `split-rest` formula nodes. The "Add Speaker" button adds complete table rows.

```
-- items is ["Ada"]
set("input/value", "Grace")
replay(button)
-- items is now ["Grace", "Ada"]
```

3.2.4 Conference Table: structural transformation

The conference table is the most complex example. It demonstrates *schema evolution* — refactoring a flat list into a structured table. Figure 10 shows the final result.

Five structural edits transform the list. The wildcard $*$ ensures every row is transformed simultaneously:

```
updateTag("speakers", "table")      -- list -> table
updateTag("speakers/*", "td")       -- items -> cells
wrapList("speakers/*", "tr")         -- wrap each cell in a ‹tr› row
wrapRecord("speakers/*/0/contact",  -- split-first
    field="source", tag="split-first")
insert("speakers/*", index=-1,      -- email column
    value=‹td split-rest(ref(sibling))›)


Before:  ‹ul›  ‹li› "Ada, ada@..." ‹/li›
               ‹li› "Grace, grace@..." ‹/li›  ‹/ul›

After:   ‹table›  ‹tr› ‹td› "Ada" ‹/td›
                      ‹td› "ada@..." ‹/td› ‹/tr›
                  ‹tr› ‹td› "Grace" ‹/td›
                      ‹td› "grace@..." ‹/td› ‹/tr›  ‹/table›
```

All edits are recorded as events in the DAG. Importantly, the "Add Speaker" button recorded in the list phase continues to work after the refactoring — see Section 3.2.6.

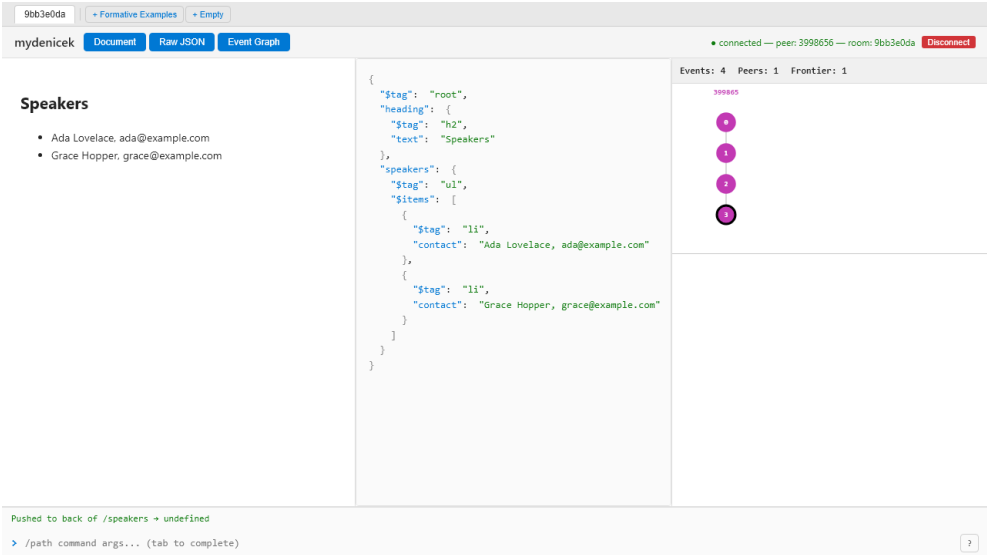

Figure 11 Initial state: both peers synced with a flat conference speaker list.

3.2.5 Conference Table: concurrent editing

Two peers start from the same conference list, disconnect, and make concurrent edits: **Alice** refactors the list into a table (as above), while **Bob** adds two new speakers via `insert`. When they reconnect and sync, Alice's wildcard edits automatically expand to include Bob's concurrently inserted items: `updateTag` changes their tags, `wrapList` wraps them in ‹`tr`› lists, and `insert` adds the split formula cells. The result is a table with all four speakers — each with correctly split name and email columns — even though Bob inserted plain ‹`li`› items into a ‹`ul`› list. This *wildcard-affects-concurrent-insertions* property (Section 2.5.4) is a direct consequence of the replay-based edit transformation approach.

Figures 11 to 14 show the four stages of this process.

3.2.6 Button replay after schema evolution

Recorded edit sequences survive structural refactoring. The "Add Speaker" button was recorded against a flat ‹`ul`› list — its steps insert a ‹`li`› item and copy the input value. After Alice refactors the list into a ‹`table`› with formula columns, clicking the button still works: each recorded step is transformed through all structural edits that happened after recording. The replayed insert produces a complete table row with split-first and split-rest cells, as if recorded against the table.

Replay reuses the same selector rewriting rules as concurrent editing. The system finds the recorded edit in the topological order, transforms it forward

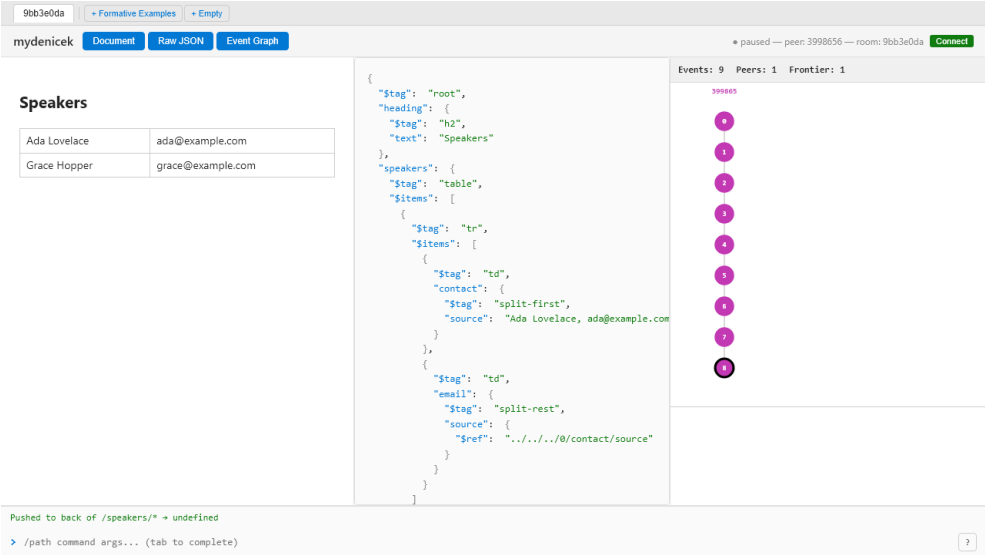

Figure 12 Alice (offline) refactors the list into a two-column table with split-first/split-rest formulas.

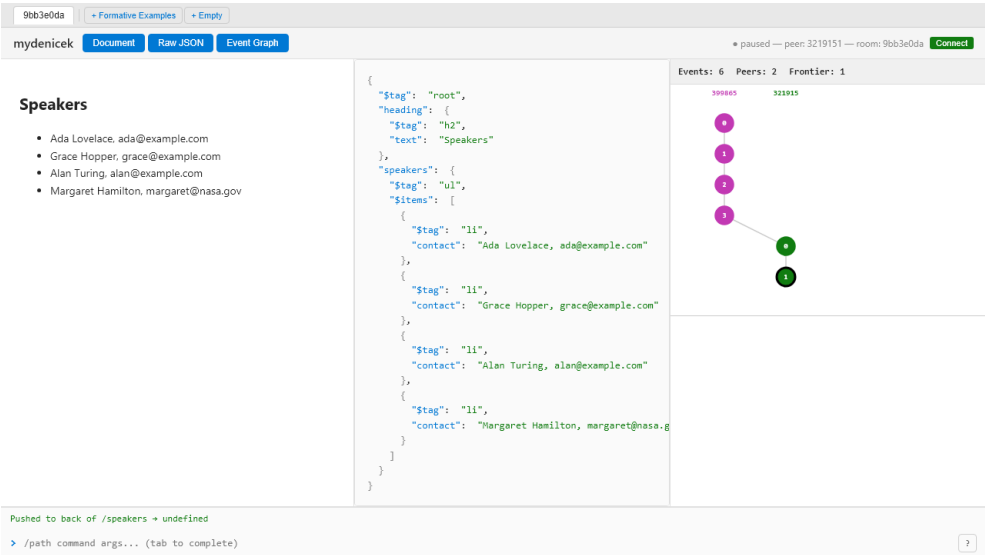

Figure 13 Bob (offline) adds two speakers to the original list structure.

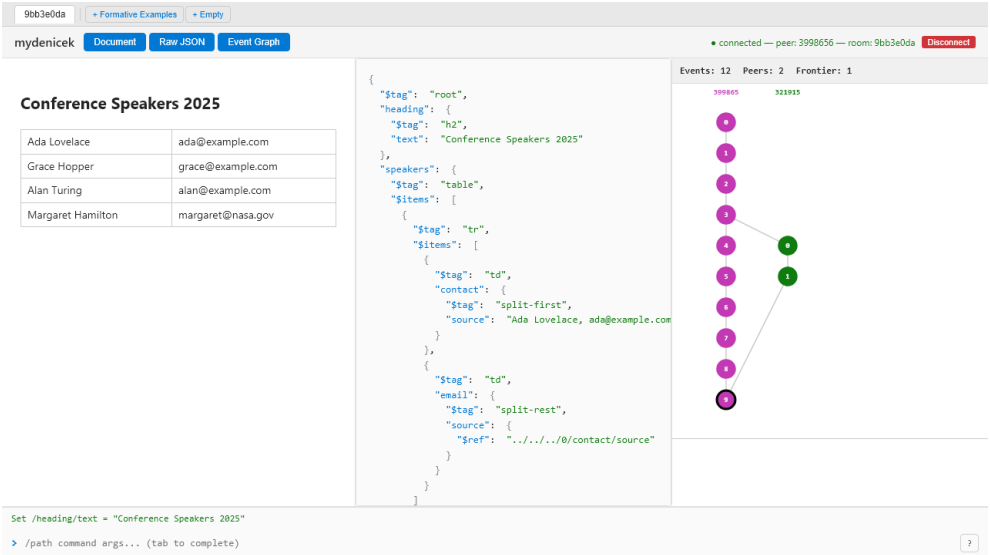

Figure 14 After merge: all four speakers appear in the table. The event graph shows the concurrent fork merging at a single commit.

through every later edit that could affect it (structural edits and wildcard-targeting edits), and commits the adjusted edit as a new event at the current frontier.

3.3 Testing strategy

Testing distributed systems is fundamentally harder than testing sequential programs: bugs arise from specific interleavings of concurrent events, message orderings, and failure patterns that are difficult to reproduce [29]. mydenicek addresses this through a layered testing strategy, organized as a testing pyramid:

- **Unit tests** (over 280 cases) test individual edit types in isolation: does a rename produce the expected document? Does an insert land at the correct index? Does a structural edit's `transformSelector` rewrite a given path correctly?
- **Integration tests** (over 40 cases) test edit *interactions*: two concurrent edits on a shared document, the full `resolveAgainst` pipeline on a small DAG, reference rewriting through structural changes, and undo/redo across peers. A dedicated *concurrent pair matrix* (`concurrent-pair-matrix.test.ts`) systematically tests one scenario per non-trivial edit-type pair — 21 pairs covering all combinations where `transformSelector` or payload rewriting produces a non-identity result (e.g., rename+insert, wrapRecord+insert, copy+edit-on-source, updateTag+rename). Each test verifies both convergence and

intention preservation.

- **Property-based tests** using `fast-check` (described in Section 3.4) randomize across all layers: they generate random edit sequences, random sync orderings, and assert convergence and intention preservation invariants. This is the highest-value layer — it exercises scenarios that hand-written tests would never cover.
- **Formative example tests** (6 cases) simulate realistic multi-peer workflows: recording, replay, formula evaluation, schema evolution, and button replay after refactoring.
- **Sync end-to-end tests** (21 cases) cover the WebSocket relay: synchronization, late join, concurrent edits, reconnection, and offline convergence.
- **Browser end-to-end tests** (Playwright) verify that two browser peers can sync edits via the deployed server, closing the loop from UI to transport to CRDT and back. These run separately from the 358-test unit suite.
- **Continuous integration** via GitHub Actions runs all layers on every push.

In total, the test suite has 358 tests (337 in the core package, 21 in the sync package) comprising approximately 7,600 lines of test code — more than the 5,800 lines of the CRDT core itself. Deno's built-in coverage tool reports **90% branch coverage** (fraction of decision branches — if/else, switch cases — taken by at least one test) **and 81% line coverage** (fraction of source lines executed) across the CRDT core. The remaining gaps are concentrated in defensive error-throwing branches. The sync package achieves 95% branch coverage on the room logic; the WebSocket transport layers are covered by Playwright browser end-to-end tests rather than unit tests.

3.4 Property-based tests

The file `tests/core-properties.test.ts` uses the `fast-check` library to randomize edit sequences, sync operations, and delivery orders, then asserts invariants on the resulting document states. This approach is a form of *randomized concurrency testing* [29]: instead of enumerating all possible interleavings (infeasible for concurrent edit operations on tree structures), the fuzzer samples random edit sequences and `fast-check`'s shrinking algorithm reduces failing cases to minimal counterexamples.

The tests run against five document schemas (flat list, flat record, nested list-of-records, deeply nested lists, document with references) and exercise all eleven edit types. The default configuration models three `Denicek` peers; a separate test suite uses five peers to verify that convergence holds beyond pairwise interactions. Operations are either local edits or pairwise sync actions; each test generates sequences of 5 to 50 operations per run.

The invariants checked are:

- **Convergence.** After a final full sync round, all peers serialize to the same JSON. This directly exercises the convergence argument of Section 2.4.
- **Idempotency.** Re-delivering an already-ingested event has no effect.
- **Commutativity.** For two disjoint remote event batches, ingesting them in either order produces the same document.
- **Associativity.** For three peers producing disjoint events, any pairwise merge order yields the same state.
- **Intent preservation.** Non-conflicting concurrent additions all appear in the merged document.
- **Out-of-order delivery tolerance.** Shuffled event delivery with the causal buffer produces the same state as causal delivery.

The property suite caught several bugs during development: wildcard-over-concurrent-insert failures and copy-then-rename retargeting errors were both discovered by shrunk counterexamples.

3.5 Performance

Table 5 reports benchmark results using Deno's built-in benchmarking framework (`deno bench`), which performs automatic warmup and reports statistical aggregates over multiple iterations.

Table 5 Benchmark results (multiple iterations with warmup). Intel i7-12700H, Deno 2.7, Windows 11.

Workload	N	Time (avg)	Min	Max
local-append	100	0.67 ms	0.36 ms	4.7 ms
local-append	2000	12 ms	8.7 ms	23 ms
sync-linear	100	1.2 ms	0.72 ms	5.8 ms
sync-linear	2000	40 ms	19 ms	61 ms
concurrent-sync	100	2.1 ms	1.2 ms	12 ms
concurrent-sync	2000	329 ms	246 ms	378 ms

local-append: single peer, sequential inserts. *sync-linear*: N events delivered causally. *concurrent-sync*: two peers edit disjoint subtrees concurrently, then sync.

For typical Denicek sessions ($N \leq 100$), average runtime is under 4 ms for all workloads. The measured scaling matches the complexity analysis of Section 2.3.2:

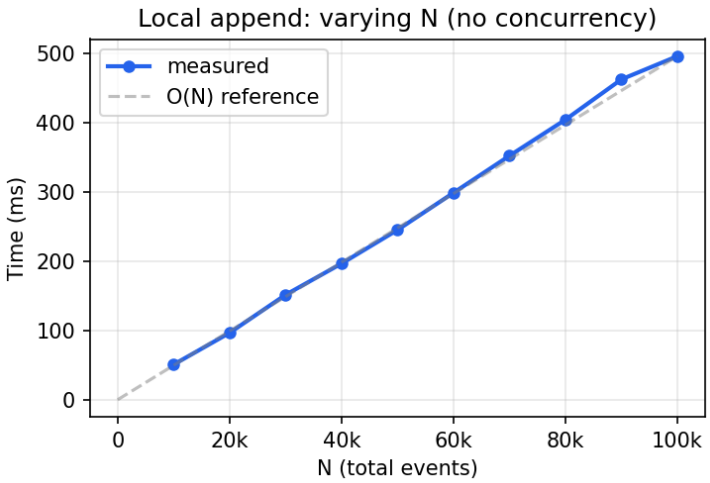

Figure 15 Local append: varying N with no concurrency ($C_{\text{total}} = 0$). Time grows linearly, confirming the $O(N)$ term.

local-append scales linearly in N (validated up to $N = 100{,}000$), while concurrent-sync with equal branches scales with $C_{\text{total}} = (N/2)^2$. Figure 15 isolates the N term by measuring local-append with no concurrency; Figure 16 shows the equal-branch concurrent case plotted against $C_{\text{total}} = N^2/4$.

3.6 Limitations

No formal proof of intention preservation. Convergence follows from the G-Set and deterministic eval (Section 2.4). However, intention preservation — the property that concurrent edits produce results matching users' intent — is validated only empirically, through formative examples and property-based tests. A formal proof (e.g., using TLA+ or VeriFx [27]) would strengthen the correctness argument but is beyond the scope of this thesis.

Materialization cost is quadratic for concurrent branches. The cost $O(N + C_{\text{total}})$ is linear for sequential editing but quadratic in the worst case ($C_{\text{total}} = O(N^2)$ for two equal-length concurrent branches). This is inherent to pairwise selector rewriting and is acceptable for the small documents and short offline intervals typical of Denicek sessions.

No character-level text editing. Primitive values (strings, numbers, booleans) are replaced atomically — concurrent edits to the same string field are resolved by last-writer-wins. Denicek operates on structured documents (trees of records, lists, and formulas), not free text, so character-level collaboration was not a priority. Supporting it would require integrating a text CRDT (such as

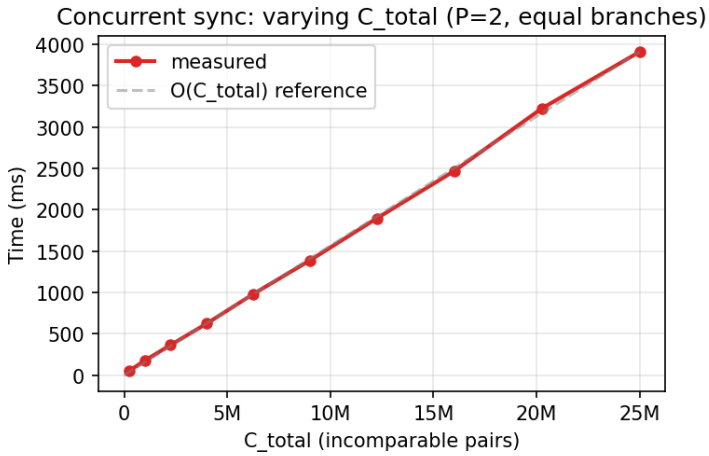

Figure 16 Concurrent sync with equal branches ($P = 2, C_{\text{total}} = N^2/4$). Plotted against C_{total}, runtime is close to linear, consistent with the dominant $O(C_{\text{total}})$ term (N co-varies as $2\sqrt{C_{\text{total}}}$).

Fugue) for primitive string values.

Chapter 4

Conclusion

This thesis set out to find a robust approach to collaborative editing for Denicek — a document-oriented end-user programming system where users program by recording and replaying edit sequences over tree-structured documents. The original Denicek relied on Operational Transformation, which is fragile and scales poorly with the number of edit types. We asked whether a CRDT-based approach could provide the same semantic richness — path-based addressing, wildcards, structural edits, programming by demonstration — while guaranteeing strong eventual consistency.

We evaluated three approaches. Two established CRDT libraries, Automerge and Loro, proved strong for general-purpose JSON collaboration but exhibited a fundamental mismatch with Denicek's requirements: their opaque unique identifiers cannot express relative paths, wildcard expansion, or replay retargeting. This evaluation identified concrete limitations — the concurrent wrap problem and the retargeting problem — that motivated a custom design.

A key contribution of this thesis is identifying the right theoretical framework for the problem. Baquero et al.'s pure operation-based CRDT framework [6] turned out to be an excellent fit: the replica state is simply a grow-only set of tagged events (a G-Set), and the document is computed by a deterministic *eval* function that replays events in topological order. This separation of concerns — append-only state for convergence, deterministic replay for semantics — made it possible to focus the design effort on intention preservation rather than on consistency mechanics. Arriving at this choice required evaluating several CRDT families and recognizing that the pure op-based model could accommodate Denicek's domain-specific edits without the pairwise transformation matrices that plague classical OT.

The central technical contribution built on this framework is **intention preservation** through selector rewriting. When the document structure changes concurrently — fields are renamed, nodes are wrapped, items are inserted or

removed — each operation must still reach its intended target and produce its intended effect. The rewriting rules ensure that references survive structural edits, wildcards expand over concurrent inserts, strict indices remain stable, and recorded edit sequences replay correctly after schema evolution. Each new edit type integrates by overriding `transformSelector` once and, where relevant, a small set of generic payload-rewriting hooks — avoiding the $O(n^2)$ pairwise rules of classical OT and keeping the system extensible.

The resulting implementation, mydenicek, is published as `@mydenicek/core`, `@mydenicek/react`, and `@mydenicek/sync` on JSR. Materialization runs in $O(N+C_{\text{total}})$ time, where N is the number of events and C_{total} is the total number of concurrent event pairs that require edit transformation. For typical sessions ($N \leq 100$) this completes under 4 ms on average. The system was validated on formative examples that exercise each requirement from the approach comparison, 358 tests (including a concurrent pair matrix and property-based convergence testing) with 90% branch coverage, and scaling benchmarks up to $N = 100{,}000$. Property-based tests provide evidence for the convergence claim beyond individual examples; benchmarks demonstrate practical feasibility. All examples are available as interactive demos at `https://krsion.github.io/mydenicek`.

Beyond the CRDT engine itself, the thesis delivers an end-to-end deployable system rather than only an algorithm description. mydenicek includes a React-based interactive frontend (mywebnicek) where users can create documents, record and replay edits, and collaborate in real time; a WebSocket sync server deployed on Azure Container Apps (chosen for its support of long-lived WebSocket sessions with scale-to-zero pricing); a CI/CD pipeline that runs the full test suite and deploys both the frontend and the sync server automatically; and a separate release pipeline that publishes packages to JSR with provenance attestation. The result is a live, publicly accessible system that reviewers and future researchers can try without local setup.

For Denicek's end users, mydenicek introduces a complementary collaboration model. The original Denicek's Git-like approach, where peers work on local branches and merge explicitly, gives users full control over how concurrent work is integrated — a valuable property in many programming workflows. mydenicek offers a different point in the design space: automatic real-time convergence that preserves user intent without requiring explicit merge steps. This positions Denicek for synchronous collaboration scenarios — shared editing sessions, live demonstrations, multi-user prototyping — where manual merging would interrupt the flow of work.

4.1 Discussion

The design involves deliberate tradeoffs. Path-based selectors make Denicek's domain-specific operations natural — wildcards, relative references, and replay retargeting all work directly on the tree structure — but this comes at the cost of a specialized design not directly reusable for general JSON collaboration. The most notable omission is character-level text editing: mydenicek uses last-writer-wins for primitive strings, whereas Automerge and Loro support fine-grained text CRDTs. For Denicek's current use cases — structured documents with formulas and recorded edit sequences — this is an acceptable tradeoff, but it limits applicability to prose-heavy collaborative documents.

The event graph architecture is clean: append-only writes, deterministic reads, no mutable shared state. However, it has inherent limitations. The grow-only DAG grows without bound, and full materialization replays the entire history. The linear extension cache mitigates this for sequential appends, but concurrent events still trigger full recomputation. These costs are manageable for the session sizes observed in practice ($N \leq 100$), but they define the boundary of the current design.

Finally, the evaluation scope supports practical viability rather than mechanical verification. The convergence proof rests on the pure op-based CRDT framework (G-Set state + deterministic eval) and is supported by property-based testing, but the selector rewriting rules that implement intention preservation have not been mechanically verified. The formative examples and concurrent pair matrix cover the interactions we identified, but they cannot exhaust all possible concurrent scenarios in an open-ended system.

4.2 Future work

The limitations above point to several directions for future work. Near-term improvements — incremental evaluation and history compaction — would extend the system's practical reach. Longer-term directions — formal verification, peer-to-peer sync, and character-level text — would broaden its applicability.

Scaling to large peer counts. The complexity analysis treats the number of peers P as a constant (2–5 peers). For larger groups, vector clock operations become $O(P)$ and start to matter. Analyzing and optimizing materialization for large P is an open problem.

Incremental eval. The linear extension cache already makes sequential appends incremental — only new events are replayed. However, after receiving concurrent remote events, `materialize` still recomputes from scratch. Incremental re-evaluation that patches only the affected portion of the document would reduce

cost for concurrent edits.

History compaction. The grow-only event DAG grows without bound. Snapshotting the materialized state and pruning old events would reduce storage and replay cost for long-lived documents and new replicas joining late.

Formal verification. Encoding the selector rewriting rules in VeriFx [27] or TLA+ would provide mechanical correctness guarantees beyond empirical testing.

Peer-to-peer sync. The current sync protocol relies on a centralized sync server. Enabling true peer-to-peer sync would remove the single point of failure and reduce latency for co-located peers.

Network-efficient encoding. Events are currently serialized as JSON. A compact binary encoding would reduce bandwidth and improve sync performance, especially for large event histories.

Fine-grained retransmission. When a single event is lost, the frontier-based catch-up resends all events that causally depend on it. A protocol that tracks individual missing events would avoid redundant retransmission.

Character-level text. Integrating a text CRDT (e.g., Fugue) for primitive strings would replace last-writer-wins semantics, though the interaction with selectors, undo, and the formula engine would require careful design.

Bibliography

[1] Tomáš Petříček, Jonathan Edwards, and Joel Jakubovic. "Design Choices of Document-Oriented Programming Systems". In: *The Art, Science, and Engineering of Programming* (2025). Submitted.

[2] Tomáš Petříček and Jonathan Edwards. "Denicek: Computational Substrate for Document-Oriented End-User Programming". In: *Proceedings of the 38th Annual ACM Symposium on User Interface Software and Technology (UIST '25)*. ACM, 2025. DOI: 10.1145/3746059.3747646.

[3] Abdessamad Imine et al. "Proving Correctness of Transformation Functions in Real-Time Groupware". In: *Proceedings of the 8th European Conference on Computer Supported Cooperative Work (ECSCW '03)*. Kluwer, 2003, pp. 277–293.

[4] Martin Kleppmann et al. "Local-First Software: You Own Your Data, in Spite of the Cloud". In: *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward! '19)*. ACM, 2019. DOI: 10.1145/3359591.3359737.

[5] Marc Shapiro et al. "Conflict-Free Replicated Data Types". In: *Proceedings of the 13th International Symposium on Stabilization, Safety, and Security of Distributed Systems (SSS 2011)*. Springer, 2011, pp. 386–400. DOI: 10.1007/978-3-642-24550-3_29.

[6] Carlos Baquero, Paulo Sérgio Almeida, and Ali Shoker. "Pure Operation-Based Replicated Data Types". In: *arXiv preprint arXiv:1710.04469* (2017). DOI: 10.48550/arXiv.1710.04469.

[7] Chengzheng Sun et al. "Achieving Convergence, Causality Preservation, and Intention Preservation in Real-Time Cooperative Editing Systems". In: *ACM Transactions on Computer-Human Interaction* 5.1 (1998), pp. 63–108. DOI: 10.1145/274444.274447.

[8] Clemens N. Klokmose et al. "Webstrates: Shareable Dynamic Media". In: *Proceedings of the 28th Annual ACM Symposium on User Interface Software & Technology (UIST '15)*. ACM, 2015, pp. 280–290. DOI: 10.1145/2807442.2807446.

[9] Clarence A. Ellis and Simon J. Gibbs. "Concurrency Control in Groupware Systems". In: *Proceedings of the 1989 ACM SIGMOD International Conference on Management of Data*. ACM, 1989, pp. 399–407. DOI: 10.1145/67544.66963.

[10] Matthias Ressel, Doris Nitsche-Ruhland, and Rul Gunzenhäuser. "An Integrating, Transformation-Oriented Approach to Concurrency Control and Undo in Group Editors". In: *Proceedings of the 1996 ACM Conference on Computer Supported Cooperative Work (CSCW '96)*. ACM, 1996, pp. 288–297. DOI: 10.1145/240080.240305.

[11] Leslie Lamport. "Time, Clocks, and the Ordering of Events in a Distributed System". In: *Communications of the ACM* 21.7 (1978), pp. 558–565. DOI: 10.1145/359545.359563.

[12] Friedemann Mattern. "Virtual Time and Global States of Distributed Systems". In: *Parallel and Distributed Algorithms: Proceedings of the International Workshop on Parallel and Distributed Algorithms*. Ed. by M. Cosnard and P. Quinton. North-Holland, 1989, pp. 215–226.

[13] Colin J. Fidge. *Timestamps in Message-Passing Systems That Preserve the Partial Ordering*. Tech. rep. Proceedings of the 11th Australian Computer Science Conference, pp. 56–66. Australian National University, 1988.

[14] Nuno Preguiça. "Conflict-free Replicated Data Types: An Overview". In: *arXiv preprint arXiv:1806.10254* (2018). DOI: 10.48550/arXiv.1806.10254.

[15] Martin Kleppmann and Alastair R. Beresford. "A Conflict-Free Replicated JSON Datatype". In: *IEEE Transactions on Parallel and Distributed Systems* 28.10 (2017), pp. 2733–2746. DOI: 10.1109/TPDS.2017.2697382.

[16] Geoffrey Litt et al. "Peritext: A CRDT for Collaborative Rich Text Editing". In: *Proceedings of the ACM on Human-Computer Interaction (CSCW '22)*. Vol. 6. 2022. DOI: 10.1145/3555644.

[17] Automerge Contributors. *Automerge: A JSON-like data structure that can be modified concurrently*. Version 2.x evaluated in this thesis (npm `@automerge/automerge` 2.2). 2024. URL: https://automerge.org/.

[18] Loro Contributors. *Loro: Reimagine state management with CRDTs*. Version 1.x evaluated in this thesis (npm `loro-crdt` 1.2). 2024. URL: https://loro.dev/.

[19] Martin Kleppmann et al. "A Highly-Available Move Operation for Replicated Trees". In: *IEEE Transactions on Parallel and Distributed Systems* 33.7 (2021), pp. 1711–1724. DOI: 10.1109/TPDS.2021.3118603.

[20] Kevin Jahns. *Yjs: A CRDT framework with a powerful abstraction of shared data*. 2024. URL: https://yjs.dev/.

[21] Joseph Gentle and Martin Kleppmann. "Collaborative Text Editing with Eg-walker: Better, Faster, Smaller". In: *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys '25)*. ACM, 2025. DOI: 10.1145/3689031.3696076.

[22] Joseph Gentle. *Diamond Types: A CRDT for text editing based on an event graph*. 2024. URL: https://github.com/josephg/diamond-types.

[23] Vadim Dalecky. *json-joy: JSON CRDT and operational transformation library*. 2024. URL: https://jsonjoy.com/.

[24] Matthew Weidner et al. "For-Each Operations in Collaborative Apps". In: *Proceedings of the 10th Workshop on Principles and Practice of Consistency for Distributed Data (PaPoC '23)*. ACM, 2023, pp. 56–62. DOI: 10.1145/3578358.3591323.

[25] Jim Bauwens. "Nested Pure Operation-Based CRDTs". In: *37th European Conference on Object-Oriented Programming (ECOOP 2023)*. Vol. 263. LIPIcs. Schloss Dagstuhl, 2023, 2:1–2:25. DOI: 10.4230/LIPIcs.ECOOP.2023.2.

[26] Jim Bauwens and Elisa Gonzalez Boix. "From Causality to Stability: Understanding and Reducing Meta-Data in CRDTs". In: *Proceedings of the 17th International Conference on Managed Programming Languages and Runtimes (MPLR '20)*. ACM, 2020. DOI: 10.1145/3426182.3426183.

[27] Kevin De Porre, Carla Ferreira, and Elisa Gonzalez Boix. "VeriFx: Correct Replicated Data Types for the Masses". In: *37th European Conference on Object-Oriented Programming (ECOOP 2023)*. Vol. 263. LIPIcs. Schloss Dagstuhl, 2023. DOI: 10.4230/LIPIcs.ECOOP.2023.9.

[28] Matthew Weidner et al. "Collabs: A Flexible and Performant CRDT Collaboration Framework". In: *arXiv preprint arXiv:2212.02618* (2023). DOI: 10.48550/arXiv.2212.02618.

[29] Ege Berkay Gulcan et al. "Model-guided Fuzzing of Distributed Systems". In: *Proceedings of the ACM on Programming Languages (OOPSLA '25)*. ACM, 2025. DOI: 10.1145/3763060.

Appendix A

Implementation repository

The mydenicek implementation evaluated in this thesis is maintained as a separate Git repository and published on JSR.

Repository. `https://github.com/krsion/mydenicek`

Published packages. `@mydenicek/core@0.5.0`, `@mydenicek/react@0.4.1`, and `@mydenicek/sync@0.1.4`.

Evaluated revision. Commit `532bdf90` is the revision against which all measurements, tests, and benchmarks reported in this thesis were produced.

Contents.

- `packages/core/` — the core CRDT engine (event graph, edits, selector rewriting, materialization). The complexity analysis in Section 2.3.2 refers to code in this package.
- `packages/sync/` — the WebSocket relay server library.
- `packages/react/` — React bindings for the core CRDT.
- `apps/mywebnicek/` — the React single-page application used for the web demos referenced in Section 3.2.
- `apps/sync-server/` — the deployed sync server.
- `packages/core/tests/` and `packages/sync/tests/` — unit, property-based, and formative tests.

Build and test. The project uses Deno 2.x. After cloning, run:

```
deno task check     # type-check + lint across workspace
deno task test      # run all tests
```

The full suite on the evaluated revision passes with 358 tests across 28 files (unit, integration, property-based, and formative), including two 5-peer convergence test suites with 1000 fuzzing runs each. Branch coverage is 90% for the core package.

Running the demos locally.

```
deno task sync-server    # relay server (ws://localhost:8080)
deno task mywebnicek:dev # web app (http://localhost:5173)
```

The live demo is at `https://krsion.github.io/mydenicek`.

Thesis repository. The Markdown-plus-LaTeX source of this thesis is at `https://github.com/krsion/master-thesis`.